

Swift: 启发式多信号融合自适应网络拥塞控制算法

杭天铨

南京邮电大学

1224045520@njupt.edu.cn

摘要

端到端网络传输的服务对象正在从固定终端和近端高质量链路,扩展到跨地域远距离传输、蜂窝/无线/卫星等多接入路径,以及手机、笔记本、台式机和边缘设备等异构终端并存的复杂场景。传统基于丢包的算法存在拥塞信号滞后与缓冲区膨胀问题,基于延迟的算法面临 RTT 测量噪声与共存公平性不足等挑战;当端点、链路、RTT 和业务条件同时变化时,固定参数、单一信号驱动的拥塞控制更难兼顾吞吐、时延与稳定性。本文提出 Swift——一种面向上述广泛端到端传输场景的启发式多信号融合自适应拥塞控制方法,其核心创新收敛为三点:(1)多信号拥塞状态感知,将 ECN 状态、拥塞避免状态机状态、拥塞事件语义、RTT 与最小 RTT、在途字节等协议栈内部信息组织为 11 维有效观测子空间(嵌入含 4 项路由元数据的 15 元素传输容器),并以两级优先级判定区分超时、ECN 触发的窗口收缩与普通丢包;(2)ns3-gym 集成的事件驱动启发式融合控制,在 GetSsThresh 与 IncreaseWindow 回调处经同步请求-应答交换动作 [$new_ssThresh, new_cWnd$],由最小 RTT 感知的 RTT 膨胀比、反馈值相对慢速基线的偏移和连续增长趋势调节每流 $\alpha \in [0.85, 1.30]$,并以有界步长逼近由滑动时间窗投递速率最大值滤波得到的 $\alpha \times BDP$ 目标窗口;(3)面向不确定拥塞信号的稳定性与安全机制,包括差异化窗口保留因子(丢包 0.70、ECN 0.75、超时 0.50)、至多 3 次连续缩减、降窗后 4 个 ACK 事件冻结、基于 $\min(cwnd, BDP)$ 的慢启动阈值更新、窗口钳位以及 CA_LOSS 下的陈旧动作失效。该控制器为确定性白盒 Python 启发式,不是端到端训练的强化学习策略。

在 ns-3.40 离散事件仿真平台上,本文以哑铃拓扑和 3 条并发长流评估 Swift。36 个命名场景是对近端高速、共享汇聚、无线/蜂窝接入、城域/跨域广域网与 LEO/GEO 卫星链路参数的剖面化模拟,并非对手机、笔记本或台式机硬件的直接建模;瓶颈为 10 Mbps~100 Gbps,基线 RTT 为 4 μ s~640 ms。纯 TCP 与 UDP 突发两种设置共形成 288 条协议记录;突发流的平均负载为瓶颈速率 32%,On 期峰值为 64%,占空比为 50%。按 ns-3 FlowMonitor 的前向 TCP 数据流统计,纯 TCP 设置下 Swift 在所考察场景中整体呈现吞吐领先,相对逐场景最强基线的增益为 2.1%~5.8% (中位数与均值均约 4.1%);Swift 严格时延最低的场景为 7 个,在 21/36 个场景中与时延最低时延相差不超过 2%,Jain 指数在 32/36 个场景中距最优值不超过 0.002,而严格丢包率最低仅见于 8 个场景。UDP 突发下,相对最强基线的吞吐增益为 2.2%~5.9% (均值 3.7%),吞吐保持率均值为 68.6%,与基线可比。代表性结果包括长途 WAN、LEO 卫星、LTE 弱覆盖和 500 Mbps 近端链路等参数剖面;混合结果则包括传统 WiFi、5G 边缘和 GEO 卫星等时延不占优的场景。公平性与丢包整体接近基线,而非统一最优。每个配置仅运行一个随机运行号,本文不作置信区间或统计显著性声明。

关键词: 拥塞控制; 启发式自适应; 多信号融合; 显式拥塞通知; 自适应参数调优; 远距离传输; 异构终端

1 引言

随着云计算、大数据、人工智能、移动互联网和物联网技术的蓬勃发展,端到端网络传输的服务对象正在从固定主机扩展到手机、笔记本、台式机、边缘节点和传感设备等异构终端,传输路径也从近端局域链路延伸到跨城、跨域、蜂窝接入、无线局域网和卫星通信等远距离复杂网络。TCP 作为互联网传输层的核心协议,其拥塞控制机制的性能直接影响网络资源利用效率、应用响应延迟和用户体验。

在这类端到端传输场景中,网络路径和终端侧条件呈现出更强的异构性。一方面,城域/广域网、蜂窝网络、WiFi 与卫星链路具有不同的 RTT 量级、丢包背景、可用带宽和链路抖动;另一方面,手机、笔记本、台式机和边缘设备等端点在处理能力、接入方式、移动性和业务负载上存在差异。同一拥塞控制策略需要适应从短 RTT 近端高速链路到长 RTT 远距离链路的变化,并应对突发跨流量、无线误码和终端负载波动。上述复杂性使传统依赖固定参数或单一拥塞信号的算法难以同时兼顾吞吐、时延和稳定性。

传统基于丢包的拥塞控制算法(如 NewReno [1]、CUBIC [2])将丢包作为拥塞信号,但丢包通常是较滞后的拥塞指示,可能伴随缓冲区占用和排队时延上升,形成 Bufferbloat 问题 [3]。Vegas [4] 等基于时延的算法尝试通过 RTT 变化提前感知拥塞,BBR [5] 则估计瓶颈带宽与传播 RTT;这类测量可能受路径抖动、终端调度和接入条件变化影响,与其他算法共存时也需专门评估公平性 [6]。

ECN 机制 [7] 允许网络设备在发生拥塞且报文支持 ECN 时进行标记,而不必仅仅依赖丢包传递信号。DCTCP [8] 利用 ECN 标记比例进行细粒度调整。不同算法对 ECN、RTT、丢包和协议栈状态的组合方式不同,如何在多样端点与路径条件下联合解释这些信号仍值得研究。

近年来,强化学习在网络拥塞控制中的应用受到关注 [9]。Aurora [10] 采用 PPO 算法训练深度神经网络策略,Orca [11] 使用强化学习动态调整 AIMD 参数;SIGCOMM 2023 的 Sage [12] 进一步将经典启发式控制知识融入学习过程,以改善策略训练和真实互联网环境中的表现。然而,这类学习型方案通常需要训练与模型推理设施,其多目标权衡和训练分布外行为需要专门验证。本文选择另一设计点:以协议栈内部信号驱动的白盒启发式规则作为决策主体,仅在参数层进行轻量反馈调整,使控制路径可逐分支检查且不依赖训练样本。

与单一网络域相比，跨地域和多接入传输的拥塞信号更难解释。丢包可能来自真实队列溢出，也可能来自无线误码或链路切换；RTT 升高既可能表示排队，也可能来自路径变化、终端调度延迟或接入链路抖动；突发 UDP/RPC/视频/备份流量还会造成短时间带宽挤占。近期发表于 IEEE/ACM ToN 的 MPLibra [13] 也表明，在异构多路径环境中结合经典控制与学习决策是一条值得探索的路线。因此，Swift 的设计目标不是某一类专用网络，而是服务远距离传输、手机/笔记本/台式机异构端点以及多样接入与路径条件的通用端到端传输。相应地，本文将微秒级 RTT 近端高速链路、无线局域网、蜂窝接入、城域/跨域广域网与卫星链路组织为覆盖不同 RTT、带宽和跨流量条件的评估集合；这些场景模拟链路参数，并不等同于对具体终端硬件的验证。

针对上述挑战，本文提出 Swift 算法——一种启发式多信号融合自适应网络拥塞控制方法。ns-3 侧在 GetSsThresh 和 IncreaseWindow 回调处通过 ns3-gym/OpenGym [14] 同步请求-应答路径交换 15 元素观测容器与二元素动作；PktsAacked、CongestionStateSet 和 CwndEvent 负责更新 RTT、拥塞状态、事件及陈旧动作状态。协议栈外的 Python 控制器据此执行确定性规则，而非端到端训练策略。本文的核心创新收敛为如下三点：

(1) 面向远距离与异构端点的多信号拥塞状态感知：将 ECN 状态、拥塞避免状态机状态、拥塞事件语义、RTT 与最小 RTT、在途字节等协议栈内部信息组织为 11 维有效观测子空间，并以两级优先级判定区分超时、ECN 触发的窗口收缩与普通丢包，为多样接入与路径条件提供联合判定依据。

(2) ns3-gym 集成的事件驱动启发式融合控制：由最小 RTT 感知的 RTT 膨胀比、性能反馈值相对其慢速基线的偏移量以及连续增长趋势共同调节每流乘性增加因子 $\alpha \in [0.85, 1.30]$ ，并以有界步长逼近 $\alpha \times BDP$ 目标窗口；BDP 由滑动时间窗口累计确认字节计算的投递速率及 40 样本最大值滤波给出，并结合最小 RTT 感知的 HyStart 风格慢启动退出和管道利用率感知增量。

(3) 面向不确定拥塞信号的稳定性与安全机制：依据信号类别施加差异化窗口保留因子（丢包 0.70、ECN 0.75、超时 0.50），并以至多 3 次连续缩减、降窗后 4 个 ACK 事件冻结、基于 $\min(cwnd, BDP)$ 的慢启动阈值更新、窗口安全钳位以及 CA_LOSS 下丢弃陈旧待应用窗口等机制，限制不确定信号和陈旧估计引起的过度调整。

2 相关工作

2.1 传统拥塞控制算法

2.1.1 基于丢包的算法

TCP Tahoe 和 Reno 是最早的拥塞控制算法 [15]，引入了慢启动、拥塞避免和快速重传/恢复机制。TCP NewReno [1] 改进了快速恢复机制，能够处理突发丢包情况，是 RFC 6582 定义的标准算法。

TCP CUBIC [2] 是 Linux 自 2.6.19 版本以来的默认

算法，采用三次函数模型调整窗口：

$$W(t) = C \cdot (t - K)^3 + W_{max} \quad (1)$$

其中 $K = \sqrt[3]{W_{max} \cdot \beta / C}$ 。CUBIC 的窗口增长与 RTT 无关，在高 BDP 网络中具有更快的收敛速度。

基于丢包的控制部分深缓冲路径上可能较晚获得拥塞信号并积累排队时延；其乘性缩减参数因算法而异，但标准丢包响应通常不会直接利用本文观测的 ECN 状态机、CA 状态和 RTT 组合来区分三类事件。

2.1.2 基于延迟的算法

TCP Vegas [4] 通过比较期望吞吐量 $cwnd/BaseRTT$ 与实际吞吐量 $cwnd/RTT$ 的差值 Δ 调整窗口，当 $\Delta < \alpha$ 时增窗， $\Delta > \beta$ 时降窗。

BBR [5] 通过估计瓶颈带宽 $BtlBw$ 和最小 RTT RTT_{prop} ，以 BDP 为目标调整发送速率： $pacing_rate = pacing_gain \times BtlBw$ 。BBR 采用周期性增益调整实现带宽探测。

Copa [16] 将目标排队延迟设为 $RTT_{standing} - RTT_{min}$ 的函数，并引入竞争模式检测机制。

这类算法面临 RTT 测量不稳定（受调度延迟、路径抖动影响）、与丢包算法共存公平性差（研究表明 BBR 与 CUBIC 共存时吞吐量可下降 30% 以上 [6]）、参数敏感等问题。

2.1.3 混合算法

Compound TCP [17] 叠加基于延迟的窗口分量，Illinois [18] 根据 RTT 调整 AIMD 参数，Westwood [19] 通过 ACK 到达率估计带宽。混合算法在一定程度上缓解了单一信号的局限性，但增加了复杂性。

2.2 ECN 机制与相关算法

ECN [7] 通过 IP 报头中的 2 比特字段传递拥塞信息，定义四种码点（Non-ECT、ECT(0)、ECT(1)、CE）。发送端设置 ECT 标记表明支持 ECN；网络设备在队列超阈值时将 ECT 改为 CE；接收端回传 ECE 标志；发送端响应后设置 CWR 标志。

DCTCP [8] 利用 ECN 标记比例 α 进行细粒度调整：

$$\alpha = (1 - g) \cdot \alpha + g \cdot F, \quad cwnd = cwnd \times (1 - \alpha/2) \quad (2)$$

其中 $g = 1/16$ 是平滑因子， F 是最近一个 RTT 内被标记的数据包比例。DCTCP 能根据拥塞程度进行比例响应，但参数静态预设。HPCC [20] 利用网络设备提供的 INT 精确拥塞信息进行控制，但需要特殊硬件支持。CoNEXT 2024 的 RECC [21] 联合 RTT、ECN 与历史突发信息进行拥塞判断，说明多信号协同仍是活跃研究方向；但其目标是高速 RDMA 网络，结论不能直接外推到通用互联网 TCP。

ECN 可在支持标记的主动队列管理器发生丢包前传递拥塞信息，并减少仅靠丢包反馈所需的重传；但其可用性取决于端点协商和路径设备配置。如何在不同网络条件下联合解释 ECN、RTT 与传输状态，仍有进一步研究空间。

2.3 基于强化学习的拥塞控制

Aurora [10] 采用 PPO 算法训练神经网络策略, 使用 10 维状态空间 (延迟梯度、延迟比率、发送比率等), 奖励函数为 $r = \text{throughput} - \delta \cdot \text{delay} - \lambda \cdot \text{loss}$ 。Orca [11] 在 AIMD 框架下用强化学习动态调整增加因子 α 和减少因子 β 。PCC [22] 定义实用函数 $U(x) = T \cdot x^\alpha - \delta \cdot L \cdot x - \beta \cdot D \cdot x$, 通过梯度上升最大化。Remy [23] 采用离线优化生成规则。

学习型方法的观测设计、奖励权衡、训练成本与分布外行为因方案而异, 不能仅凭状态维数判断优劣。Swift 不采用学习型策略, 而是在状态表示中引入本实现可读取的协议栈内部信号, 以确定性启发式规则完成窗口决策, 仅在参数层使用基线相对的反馈调整; 本文不主张 Swift 在表示能力上优于深度策略, 二者的取舍差异见第5节。

2.4 参数化白盒控制与带外参数优化

与“用学习模型替换控制逻辑”不同, 另一类工作把学习限制在参数层面。Gemini [24] 采用分而治之框架: 其参数化拥塞控制类 Fusion 以 Linux 内核模块形式实现固定的白盒控制逻辑 (融合窗口式与速率式增长, 以丢包率阈值 l 和 RTT 膨胀阈值 δ 作为拥塞信号, 并以 $\alpha \cdot R^{\max} T^{\min}$ 为窗口目标), 其在线优化引擎 Booster 则运行在用户态集群中, 按分钟级时间窗口聚合流级统计量, 并用贝叶斯优化在隐变量 (区域、ISP、时段) 分组内搜索控制参数 $\theta = \{\omega, \alpha, \gamma, \lambda, l, \delta, n\}$ 。该设计的优势是数据面开销与传统算法相当, 且已在生产网络完成大规模部署验证。

Swift 与之处于不同的设计点。第一, **决策位置与时间尺度**: Gemini 的控制逻辑固定在内核数据面内, 学习 (贝叶斯优化) 只在带外、以分钟级和流组粒度更新参数; Swift 把整个决策策略置于协议栈之外, 并在每个 ACK 事件 (GetSsThresh/IncreaseWindow 回调) 上被同步调用, 因此其自适应发生在单流、单事件粒度上。第二, **可用信号**: 带外优化器只能观察流级统计量, 而 Swift 的状态向量直接来自协议栈内部, 可读取 ECN 状态机与拥塞避免状态机, 从而区分 ECE/CWR 触发的收缩与真实丢包。第三, **部署代价**: 这一选择的代价是每事件一次跨进程往返, 本文实现依赖 ZeroMQ 同步通信, 在当前形态下适用于仿真与研究平台, 而非生产数据面; Gemini 的内核态 Fusion 在这一维度上明显更易部署。第四, **证据强度**: Gemini 给出了生产网络 A/B 测试结果, 本文的证据仅来自 ns-3.40 仿真。因此, 本文将 Swift 定位为“协议栈外每事件启发式决策 + 协议栈内多信号感知”这一设计点的可行性与代价评估, 而非对 Gemini 式部署路线的替代。

3 Swift 算法设计

3.1 系统架构

Swift 的系统架构由四个核心模块组成, 整体控制回路如图1所示。**网络仿真模块**基于 ns-3 [25] 构建, 实现完整 TCP/IP 协议栈和多种网络拓扑 (近端高速链路、哑铃型、无线/移动、广域网与卫星链路等)。**状态采集与决策应用模块**在 TCP 协议栈的五个关键回调点

(GetSsThresh、IncreaseWindow、PktsAacked、Congestion-StateSet、CwndEvent) 采集 11 维状态参数, 并将决策模块输出的 ssthresh/cwnd 决策应用于 TCP 连接 (该模块基于 ns3-gym/OpenGym [14] 环境框架实现, 仅作为跨进程状态—决策传输通道, 不含任何学习逻辑)。**启发式决策模块**采用 Python 实现, 执行多信号融合拥塞检测、差异化拥塞响应、自适应参数调优和融合控制策略计算, 维护每个 TCP 连接的独立状态。**进程间通信模块**采用 ZeroMQ 消息队列和 Protocol Buffers 序列化, 实现仿真进程与决策进程间的同步请求—应答交互; 由于该路径依赖 ZMQ 同步通信, 当前实验未启用 MTP 并行仿真。

Swift 的决策问题可描述为一个事件驱动决策回路: 在 GetSsThresh 与 IncreaseWindow 回调上, 以当前状态向量 s (11 维有效状态, 嵌入于 15 元素传输容器, 另含 4 项元数据通道) 为输入, 由确定性规则输出 $\langle \text{new_ssThresh}, \text{new_cWnd} \rangle$ 。性能反馈值 r_t 由环境按确认进度与 RTT/拥塞代价合成, 仅用于参数层的慢速调整 (见第3.5节), 不构成策略训练信号。

状态向量设计遵循以下原则: (1) **实现相关性**——覆盖当前规则使用或记录的 ECN 状态、CA 状态、窗口、在途字节和 RTT 等协议栈信息; (2) **可观测性**——参数均由 ns-3 回调接口提供; (3) **有界处理**——观测维数和每流历史缓冲区均固定, 便于控制单次规则计算与状态开销。

3.2 11 维观测空间设计

Swift 的状态向量采用“11 维有效状态子空间 + 4 项元数据通道”的 15 元素 uint64 传输布局, 如表1所示。连接标识 (socketUuid、nodeId) 用于多流识别和状态隔离, envType 与 simTime 用于环境标识和投递速率计算; 窗口状态 (ssThresh、cWnd)、传输状态 (segmentSize、segmentsAacked、bytesInFlight)、时延测量 (lastRtt、minRtt) 以及回调与拥塞控制状态 (calledFunc、caState、caEvent、ecnState) 构成其余字段。前 4 项是跨进程路由与时序元数据; 后 11 项被传递给决策模块, 其中当前规则直接读取除 caEvent 外的字段, caEvent 作为协议栈事件记录保留。因此, 本文所称“11 维观测”是传输容器的有效状态子空间, 而非声称每一字段均进入当前版本的每个决策分支。

表 1: Swift 11 维有效观测子空间 (对应 15 元素容器索引 4~14)

索引	参数	说明
4	ssThresh	慢启动阈值 (B)
5	cwnd	拥塞窗口 (B)
6	segSize	段大小 (B)
7	segmentsAacked	确认段数
8	bytesInFlight	在途字节数
9	lastRtt	最近 RTT(μs)
10	minRtt	最小 RTT(μs)
11	calledFunc	回调类型 (0/1)
12	caState	CA 状态 (0-4)
13	caEvent	CA 事件 (0-7)
14	ecnState	ECN 状态 (0-5)

状态设计中的三个协议栈内部字段为 caState、

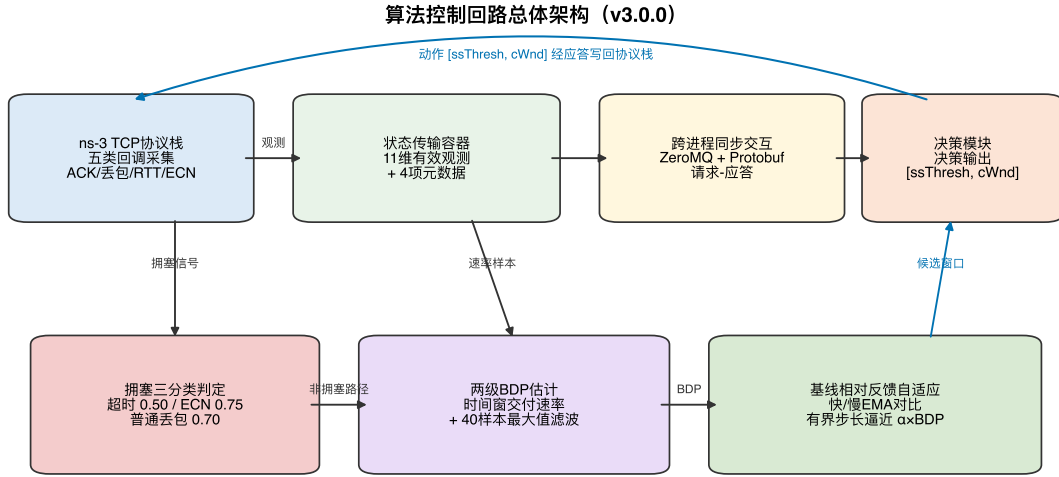


图 1: Swift 控制回路总体架构: 协议栈回调采集的状态向量经跨进程同步信道送入启发式决策模块, 依次执行拥塞三分类判定、两级 BDP 估计与基线相对反馈自适应, ssthresh/cwnd 决策经应答写回协议栈

caEvent 和 ecnState。**caState** 反映 TCP 拥塞控制状态机的当前状态: CA_OPEN (0, 正常传输)、CA_DISORDER (1, 收到重复确认)、CA_CWR (2, 响应 ECN 缩减窗口)、CA_RECOVERY (3, 快速恢复中)、CA_LOSS (4, 重传超时后的丢包状态)。**caEvent** 记录 TX_START、CWND_RESTART、COMPLETE_CWR、LOSS、ECN_NO_CE、ECN_IS_CE 以及延迟/非延迟 ACK 事件 (枚举值 0-7)。**ecnState** 反映 ECN 协商和标记状态: ECN_DISABLED (0)、ECN_IDLE (1)、ECN_CE_RCVD (2)、ECN_SENDING_ECE (3)、ECN_ECE_RCVD (4)、ECN_CWR_SENT (5)。

这些字段使协议栈内部语义可随窗口、RTT 与在途字节等状态一并传输。当前实现的三分类分支直接使用 calledFunc、caState 和 ecnState; caEvent 由 CwndEvent 更新并保留在观测中。整个 Python 决策路径不包含神经网络推理。

3.3 多信号融合拥塞检测

Swift 将拥塞信号分为三类, 采用两级优先级层次结构进行综合判断, 并引入连续递减保护机制防止窗口过度收缩。

第一优先级——窗口收缩回调检测: 当 `calledFunc = 0` 时 (GetSsThresh 回调), Python 规则按实现顺序解释收缩原因: 若 `caState = CA_LOSS`, 归类为超时; 否则, 若 `ecnState ∈ {ECN_CE_RCVD, ECN_ECE_RCVD}` 或 `caState = CA_CWR`, 归类为 ECN; 其余情况归类为普通丢包。该顺序是代码中的确定性分类规则, 不依赖未经测量的信号似然比。

第二优先级——ECN 状态直接检测: 在 IncreaseWindow 路径上, 当 `ecnState ∈ {ECN_CE_RCVD, ECN_ECE_RCVD}` 时, 规则同样进入 ECN 响应。该分支要求瓶颈队列实际执行 CE 标记; 若队列不标记 CE, 则不会观察到相应状态。第4.1节中的实验统一使用启用 ECN 标记的 RED 队列,

并对四种协议对称开启 ECN。

连续递减保护机制: Swift 维护连续递减计数器 d 。拥塞响应使 d 自增; 降窗后的冻结路径不清零该计数器, 冻结结束后进入非拥塞窗口更新路径时再重置。当 $d > D_{max} = 3$ 时, 规则保持当前窗口并仍设置 4 个 ACK 事件的冻结计数, 从而限制连续信号导致的进一步乘性缩减。以普通丢包保留因子 0.70 为例, 前三次响应后的比例为 $0.70^3 \approx 34.3\%$; 该数值仅说明代码的累积缩减幅度, 不构成优于其他算法的理论证明。

完整的多信号融合拥塞检测算法如算法1所示。

Algorithm 1 Multi-Signal Fusion Congestion Detection

Require: Observation *obs*, consecutive decrease counter d
Ensure: (*is_congested*, *type*)

```

1: // Level-1: window-reduction callback (GetSsThresh)
2: if obs.calledFunc = 0 then
3:   if obs.caState = CA_LOSS then
4:     return (True, TIMEOUT)
5:   else if obs.ecnState ∈ {CE_RCVD, ECE_RCVD} or
      obs.caState = CA_CWR then
6:     return (True, ECN)
7:   else
8:     return (True, LOSS)
9:   end if
10: end if
11: // Level-2: ECN state inspection
12: if obs.ecnState ∈ {CE_RCVD, ECE_RCVD} then
13:   return (True, ECN)
14: end if
15: return (False, NONE)
  
```

3.4 差异化拥塞响应机制

NewReno、CUBIC 等丢包型算法通常以固定的乘性缩减规则响应拥塞, 不同算法的具体保留因子并不相同, 但其标准响应不会按本文所用的超时、普通丢包与 ECN 三类语义分别选择参数。Swift 在当前实现中采用如下差异化窗口保留因子:

$$new_cwnd = \rho \cdot cwnd,$$

$$\rho = \begin{cases} 0.70 & \text{丢包 (LOSS)} \\ 0.75 & \text{ECN 标记} \\ 0.50 & \text{超时 (TIMEOUT)} \end{cases} \quad (3)$$

这组参数体现一种工程化的严重程度排序：RED 的 CE 标记可在丢包前出现，因此采用 0.75 的相对温和收缩；普通丢包采用 0.70；CA_LOSS 所表示的重传超时采用 0.50 并重新进入慢启动。上述数值是当前实现的手工预设，而非由似然估计或最优性证明得到；仓库也未保存分别关闭这些机制的消融实验，因此本文仅将其解释为实现设计。

慢启动阈值更新为 $new_ssthresh = \max(\rho \cdot \min(cwnd, BDP), new_cwnd)$ ，窗口下限为 $4 \times MSS$ 。该写法将阈值参考量限制在当前窗口与 BDP 估计的较小者，并确保阈值不低于新窗口；其效果需结合后续实验观察，而不被表述为一般性最优结论。

此外，当前实现在拥塞响应路径上还包含三项与信号不确定性直接相关的安全约束。(1) **降窗后冻结**：每次进入拥塞响应即将冻结计数器置为 4 个 ACK 事件，且该赋值先于连续递减下限判定执行，因此即使响应被 D_{max} 抑制，队列仍获得排空窗口。(2) **超时重入慢启动**：当拥塞类型为超时，除施加 $\rho = 0.50$ 外还重置慢启动标志与慢启动阶段最小 RTT 采样，使控制律回到指数上探而非在错误的 BDP 估计附近继续微调。(3) **陈旧动作丢弃**：由于 GetSsThresh 回调接收的是常量协议栈状态，Python 侧返回的 $cwnd$ 需缓存至下一次 IncreaseWindow 才能生效；若在此期间协议栈进入 CA_LOSS（重传超时已重置窗口），环境侧将直接丢弃该缓存动作，避免陈旧决策覆盖协议栈的丢包恢复过程。差异化响应算法如算法2所示。

3.5 启发式反馈自适应的参数调优

Swift 根据多种网络状态因素动态调整乘性增加因子 $\alpha \in [0.85, 1.30]$ （初始值 1.10），控制拥塞避免阶段窗口增长速度。当前实现的调优逻辑基于 RTT 膨胀、反馈值 EMA 和连续增长趋势三个启发式维度。与固定阈值不同，RTT 膨胀阈值随最小 RTT 增大而放宽，其设计意图是降低长 RTT 路径在 BDP 填充阶段过早收缩的可能性。

基于最小 RTT 感知阈值的调整：设 $r = lastRtt/minRtt$ 。Swift 首先根据 $minRtt$ 计算松弛量 $slack = 0.3 + 0.15\sqrt{\max(0, minRtt_{ms} - 1)}$ ，并构造 $low = 1 + slack$ 、 $mid = 1 + 2slack$ 、 $high = 1 + 4slack$ 三个动态阈值。当 $r < low$ 时，说明队列驻留较低， α 按 0.02 或 0.03 小步增加；当 $low \leq r < mid$ 时， α 仅增加 0.01 以维持温和探测；当 $r > high$ 时，说明排队膨胀明显， α 按 0.02 或 0.03 收缩并重置连续增长计数器。

基于反馈值基线相对比较的调整：同时维护性能反馈值 r_t 的快速指数移动平均 $\bar{r}_t = (1 - \eta)\bar{r}_{t-1} + \eta r_t$ ($\eta = 0.15$) 与慢速基线 $b_t = (1 - \eta_b)b_{t-1} + \eta_b r_t$ ($\eta_b = 0.02$)。仅当 $\bar{r} > b + m$ 时令

Algorithm 2 Differentiated Congestion Response

Require: Current $cwnd$, congestion type $type$, consecutive decrease counter d , BDP estimate bdp

Ensure: New window new_cwnd , new threshold $new_ssthresh$

```

1:  $d \leftarrow d + 1$ ; consecutive increases  $\leftarrow 0$ 
2:  $freeze \leftarrow 4$  {Post-decrease freeze, armed before the floor test}
3: if  $d > D_{max}$  then
4:    $new\_cwnd \leftarrow \max(cwnd, 4 \times MSS)$  {Freeze window}

5:    $new\_ssthresh \leftarrow new\_cwnd$ 
6:   return ( $new\_cwnd, new\_ssthresh$ )
7: end if
8: if  $type = \text{LOSS}$  then
9:    $\rho \leftarrow 0.70$ 
10: else if  $type = \text{ECN}$  then
11:    $\rho \leftarrow 0.75$ 
12: else if  $type = \text{TIMEOUT}$  then
13:    $\rho \leftarrow 0.50$ ; re-enter slow start
14: end if
15:  $new\_cwnd \leftarrow \max(\rho \times cwnd, 4 \times MSS)$ 
16:  $new\_ssthresh \leftarrow \max(\rho \times \min(cwnd, bdp), new\_cwnd)$ 

17: return ( $new\_cwnd, new\_ssthresh$ )
```

$\alpha += 0.01$ ，当 $\bar{r} < b - 4m$ 时令 $\alpha -= 0.01$ ，其中 $m = \max(0.25, 0.1|b|)$ 。环境在 IncreaseWindow 事件上定义 $r = \min(0.5 \text{ segmentsAcked}, 5) - p_{rtt}$ ；仅当 $lastRtt/RTT_{min} > 1.5$ 时， $p_{rtt} = \min(0.3(lastRtt/RTT_{min} - 1.5), 3)$ 。在 GetSsThresh 回调上，ECN、CA_LOSS 和其他丢包分别赋值 -3、-20 和 -10。因此，调节条件主要比较快速 EMA 与慢速基线的相对偏移，同时动态裕量仍随基线绝对值变化。

基于连续增长趋势的调整：窗口连续增长超过 8 次时，规则令 $\alpha += 0.01$ 。所有调整均限制在 $[0.85, 1.30]$ 内，以约束无线、蜂窝和广域网参数剖面下的调整幅度；该边界是预设安全范围，而非经消融证明的最优区间。完整的 α 自适应过程如算法3所示。

3.6 融合控制策略

Swift 将 BDP 估计、目标窗口渐进逼近和安全边界约束相结合。BDP 估计采用窗口最大值滤波器 (Windowed Max-Filter)：投递速率由滑动时间窗口内的累计确认字节数计算， $DR = \Delta \text{bytes}_{acked} / \Delta t$ ，时间窗口跨度取 $\min(\max(2 \times RTT_{min}, 5 \text{ ms}), 1 \text{ s})$ ，使速率反映 ACK 时钟（即瓶颈排空速率）而非单个 ACK 的载荷（后者会按每 RTT 的 ACK 数量成倍低估带宽）；速率样本存入容量为 $W_{bw} = 40$ 的滑动窗口队列，取窗口内最大值 BW_{max} 作为瓶颈带宽估计；RTT 采样窗口容量为 $W_{rtt} = 40$ ，并持续跟踪全局最小 RTT。BDP 计算为 $BDP = BW_{max} \times RTT_{min}$ ；当 BDP 估计无效时回退至当前窗口值。BDP 估计算法如算法4所示。

慢启动阶段 ($cwnd < ssthresh$)：目标窗口 $target = 2 \times BDP$ （下限 $10 \times MSS$ ）。Swift 采用 HyStart 风格退出规则：当当前 RTT 超过慢启动阶段最小 RTT 的动态阈值时结束慢启动；阈值按最小 RTT 分段取 1.25、1.30 或 1.40，其设计意图是在短 RTT 路径上较早响应 RTT 膨胀，并在长 RTT 路径上保留更宽松

Algorithm 3 RTT- and Reward-Aware Adaptive Parameter Tuning

Require: Current α , observation obs , consecutive increase count g , reward EMA $\bar{r}$, reward baseline b

Ensure: Updated α

```

1:  $r \leftarrow obs.lastRtt / obs.minRtt$ 
2:  $slack \leftarrow 0.3 + 0.15\sqrt{\max(0, obs.minRtt_{ms} - 1)}$ 
3:  $low \leftarrow 1 + slack; mid \leftarrow 1 + 2slack; high \leftarrow 1 + 4slack$ 
4: if  $r < low$  then
5:    $\alpha \leftarrow \alpha + (obs.minRtt_{ms} > 5?0.02 : 0.03); g \leftarrow g + 1$ 
6: else if  $r < mid$  then
7:    $\alpha \leftarrow \alpha + 0.01$ 
8: else if  $r > high$  then
9:    $\alpha \leftarrow \alpha - (obs.minRtt_{ms} > 5?0.03 : 0.02); g \leftarrow 0$ 
10: end if
11:  $m \leftarrow \max(0.25, 0.1|b|)$ 
12: if  $\bar{r} > b + m$  then
13:    $\alpha \leftarrow \alpha + 0.01$ 
14: else if  $\bar{r} < b - 4m$  then
15:    $\alpha \leftarrow \alpha - 0.01$ 
16: end if
17: if  $g > 8$  then
18:    $\alpha \leftarrow \alpha + 0.01$ 
19: end if
20:  $\alpha \leftarrow \text{clamp}(\alpha, 0.85, 1.30)$ 

```

Algorithm 4 Windowed Max-Filter BDP Estimation

Require: Flow state $state$, observation obs

Ensure: BDP estimate

```

1:  $state.bytes_{acked} \leftarrow state.bytes_{acked} + obs.segAcked \times obs.segSize$ 
2:  $W \leftarrow \min(\max(2 \times RTT_{min}, 5\text{ms}), 1\text{s})$  {Rate window span}
3:  $dr \leftarrow \Delta bytes_{acked} / \Delta t$  over the last  $W$  {Delivery rate}
4:  $state.bw\_samples.append(dr)$  {Sliding window, capacity  $W_{bw} = 40$ }
5:  $BW_{max} \leftarrow \max(state.bw\_samples)$ 
6:  $RTT_{min} \leftarrow \min(state.min\_rtt, obs.minRtt)$ 
7:  $bdp \leftarrow BW_{max} \times RTT_{min}$ 
8: if  $bdp \leq 0$  then
9:    $bdp \leftarrow \max(obs.cwnd, 1)$  {Fallback}
10: end if
11: return  $bdp$ 

```

的填充空间。标准增量为 $segmentsAcked \times MSS$ ；仅当 $cwnd < 0.3BDP$ 且 $lastRtt < 1.2RTT_{min}$ 时，增量提升为 $2 \times segmentsAcked \times MSS$ 。

拥塞避免阶段 ($cwnd \geq ssthresh$): 当前实现先计算目标窗口 $target = \alpha \times BDP$ (BDP 无效时回退到当前窗口)，再以有界步长向目标靠拢。当 $target > cwnd$ 时，窗口按 $\max(\gamma \times MSS, MSS, (target - cwnd)/16)$ 上探，并以 $target + \gamma \times MSS$ 为本次上限；该规则以当前差值的 1/16 作为候选步长，但不宣称固定在 16 个 ACK 事件内完成收敛。当 $target < cwnd$ 时，每个 ACK 事件按 $\max((cwnd - target)/2, MSS)$ 回落且不低于 $target$ ；当二者相等时，按 $\gamma \times MSS$ 执行每事件加性增长。当前实现取 $\gamma = 1.0$ 。若降窗冻结计数器非零，本次事件直接返回当前窗口并将计数器减一。

管道利用率感知增量与安全边界: 管道利用率 $u = bytesInFlight/cwnd$ 用于识别未充分填充的链路。该增量仅在 $lastRtt < 1.3RTT_{min}$ 时生效：当 $u < 0.4$ 时，短 RTT 路径追加 $2 \times MSS$ ，最小 RTT 超过 5 ms 的长 RTT 路径追加 $1 \times MSS$ ；当 $0.4 \leq u < 0.5$ 时，仅短 RTT 路径追加 $1 \times MSS$ 。这一分级规则用于限制长 RTT 路径上的额外增量。最终窗口被约束在 $[4 \times MSS, \max(4 \times BDP, 200 \times MSS)]$ 内，以限制 BDP 估计陈旧时的窗口范围。融合控制策略的完整算法如算法 5 所示。

3.7 每流独立状态管理

Swift 为每个 TCP 连接维护独立状态，通过 socketUid 为键的哈希表实现 $O(1)$ 查找。每流状态包括四个子模块：(1) **带宽与 RTT 估计模块**——容量 $W_{bw} = 40$ 的投递速率滑动窗口队列和容量 $W_{rtt} = 40$ 的 RTT 采样队列，支持窗口最大值带宽估计、全局最小 RTT 追踪和慢启动阶段最小 RTT 采样；(2) **拥塞控制计数器**——连续递减计数器 d 、连续增长计数器 g 、累计丢包和 ECN 事件计数，以及降窗后的 ACK 冻结计数；(3) **性能指标聚合**——吞吐量指数移动平均、当前 BDP 估计、上一步窗口和时间戳；(4) **自适应参数**—— α (初始 1.10)、 γ (默认 1.0) 和慢启动标志位。

状态管理遵循延迟初始化 (首次访问时创建) 和增量更新 (仅更新变化字段) 两个原则。每流独立管理避免了多流并发场景下的流间干扰，支持差异化控制。反馈值 EMA (平滑因子 $\eta = 0.15$) 与慢速基线 ($\eta_b = 0.02$) 由每个连接对应的决策实例独立维护，避免流间反馈串扰。

4 实验评估

4.1 仿真配置与实验矩阵

实现环境: 仿真基于 ns-3.40 [25] 与 ns3-gym/OpenGym [14]。Python 进程执行 Swift 启发式控制，C++ 环境负责观测、动作应用、拓扑、业务和队列配置。现有实验记录不足以独立核验主机、操作系统与 Python 依赖版本，因此本文不报告这些版本，也不作运行时开销比较。

拓 扑 与 负 载: 使 用 PointToPointDumbbellHelper 构造哑铃拓

Algorithm 5 Hybrid Rate-Window Fusion Control

Require: Observation obs , parameter α , BDP estimate bdp , freeze counter f

Ensure: New window new_cwnd

```
1: if  $f > 0$  then
2:    $f \leftarrow f - 1$ ;
3:   return  $obs.cwnd$  {Post-decrease freeze}
4: end if
5:  $d \leftarrow 0$  {Reset after freeze on the non-congestion path}
6: if  $obs.cwnd < obs.ssthresh$  and flow is in slow start then
7:   Apply HyStart-style RTT-inflation exit (1.25/1.30/1.40 by  $RTT_{min}$ )
8:    $target \leftarrow \max(2 \times bdp, 10 \times MSS)$ 
9:    $\Delta \leftarrow obs.segmentsAked \times MSS$ 
10:  if  $obs.cwnd < 0.3 \times bdp$  and  $obs.lastRtt < 1.2 RTT_{min}$  then
11:     $\Delta \leftarrow 2 \times obs.segmentsAked \times MSS$ 
12:  end if
13:   $new\_cwnd \leftarrow \min(obs.cwnd + \Delta, target)$ 
14: else
15:    $target \leftarrow \alpha \times bdp$ 
16:   if  $target > obs.cwnd$  then
17:      $step \leftarrow \max(\gamma \times MSS, MSS, (target - obs.cwnd)/16)$ 
18:      $new\_cwnd \leftarrow \min(obs.cwnd + step, target + \gamma \times MSS)$ 
19:   else if  $target < obs.cwnd$  then
20:      $new\_cwnd \leftarrow \max(obs.cwnd - \max(\frac{obs.cwnd - target}{2}, MSS), target)$ 
21:   else
22:      $new\_cwnd \leftarrow obs.cwnd + \gamma \times MSS$ 
23:   end if
24: end if
25: if  $obs.lastRtt < 1.3 RTT_{min}$  then
26:    $u \leftarrow obs.bytesInFlight/obs.cwnd$ 
27:   if  $u < 0.4$  then
28:      $new\_cwnd \leftarrow new\_cwnd + (RTT_{min} > 5\text{ ms} ? MSS : 2 \times MSS)$ 
29:   else if  $u < 0.5$  and  $RTT_{min} \leq 5\text{ ms}$  then
30:      $new\_cwnd \leftarrow new\_cwnd + MSS$ 
31:   end if
32: end if
33:  $new\_cwnd \leftarrow \text{clamp}(new\_cwnd, 4 \times MSS, \max(4 \times bdp, 200 \times MSS))$ 
34: return  $new\_cwnd$ 
```

扑, 左右各 3 个叶子节点 ($nLeaf = 3$) 经接入链路汇聚到一条共享瓶颈链路; 每对叶子节点上运行一条 BulkSendHelper 长流, 3 条流分别在 0.1、0.2 与 0.3 s 启动, 并统一在 20.1 s 停止, 业务持续时间参数为 20 s。MTU 为 1500 B, 应用数据单元为 1440 B, 并启用 SACK; 瓶颈队列长度为 $Q = \max(BDP \cdot nLeaf/MTU, 100)$ 个报文, 其中 BDP 由瓶颈速率与完整基础 RTT 计算。

队列管理与 ECN: 瓶颈链路两端网关对称安装 RED 队列, 标记下限 $MinTh = 0.3Q$ 、上限 $MaxTh = 0.9Q$ 、UseEcn=true, 并对四种协议统一设置 TcpSocketBase::UseEcn=On。

A/B 对照设置: 两组实验共享相同的场景参数、拓扑与随机运行号, 每个配置执行一次确定性运行, 仅 enable_udp_burst 取值不同:

- **纯 TCP 设置:** 瓶颈上仅承载 3 条 TCP 长流;
- **UDP 突发设置:** 在同一瓶颈上叠加一路 OnOffUDP 源。该源以 1024 B 报文按 On 0.5 s / Off 0.5 s 的 50% 占空比运行, On 期峰值速率取当前瓶颈速率的 64%, 长期平均负载为瓶颈速率的 32%。

场景矩阵: 共 36 个场景, 覆盖微秒级 RTT 近端高速链路、不同收敛比与拥塞程度的共享汇聚链路、非对称接入、WiFi 命名配置、5G/LTE 命名配置、城域/长途/跨域 WAN 以及 LEO/GEO 卫星链路; 瓶颈速率 10 Mbps~100 Gbps、基线 RTT 4 μ s~640 ms。每个场景在两种设置下各运行 4 种协议, 合计 $36 \times 4 \times 2 = 288$ 次仿真。场景名仅表示点到点链路的带宽与时延参数剖面; 仿真未建立手机、笔记本或台式机硬件模型, 也未使用真实 WiFi、LTE、5G 或卫星协议栈, 因而只支持路径条件层面的结论。下文以 19 个代表性场景 (S1-S19, 表2) 展开, 其余场景在第4.3.2节概述。

对比算法与评测指标: 对比对象为 NewReno [1]、CUBIC [2] 与 BBR [5]。指标包括 3 条前向流的聚合有效吞吐量 (Goodput, Mbps)、各前向流逐包平均单向端到端时延的非加权算术平均 (ms)、按同样口径计算的抖动、汇总丢包率 (%) 以及基于 3 条前向流吞吐量计算的 Jain 公平性指数 [26]。

4.2 FlowMonitor 测量口径

每条 TCP 连接在 FlowMonitor 中登记为前向数据流 (10.1.x→10.2.x, 接收端口 5000) 与反向 ACK 流。为避免把 ACK 字节和 ACK 时延混入数据面指标, 本文仅选择 3 条前向 TCP 数据流: 吞吐量为 3 条前向流各自 Goodput 之和; 场景时延与抖动先在每条流内按数据包求均值, 再对 3 条流的均值作非加权算术平均; 丢包率由前向流丢包数与发送包数汇总计算; Jain 指数由 3 条前向流的 Goodput 计算。

测量检查覆盖 36 场景、4 协议和 2 设置形成的 288 次仿真。每次仿真均包含 3 条前向 TCP 流; 指标满足吞吐量不超过瓶颈、时延不低于配置的基础单向时延以及 Jain 指数处于合法区间等约束。每个 (场景, 协议, 设置) 三元组只有一个随机运行号, 本文不报告置信区间或统计显著性, 并将结论限制在所考察的 ns-3.40 配置范围内。

表2给出详细展开的 19 个代表性场景配置。为便

于组织讨论，按路径特征将 19 个场景归为四组：**G1** 为微秒级 RTT 近端高速链路，**G2** 为近端共享/城域与跨域链路（100 Mbps~1 Gbps，含浅缓冲与深缓冲两类），**G3** 为无线与蜂窝接入链路，**G4** 为 GEO 卫星链路。

表 2：代表性 19 个场景配置（其余 17 个补充场景见正文第 4.3.2 节）

编号	组	场景	瓶颈 (Mbps)	基础单向时延 (ms)
S1	G1	intra_rack_10g	10000	0.004
S2	G1	intra_rack_25g	25000	0.004
S3	G1	leaf_spine_20g	20000	0.009
S4	G2	asymmetric_high	1000	0.012
S5	G2	congested_heavy	500	0.009
S6	G2	symmetric_low	1000	0.030
S7	G2	dc_500m	500	0.009
S8	G2	dc_100m	100	0.009
S9	G2	cross_dc_wan [†]	1000	5.020
S10	G2	wan_metro	1000	2.200
S11	G3	wifi_ac [†]	400	7.000
S12	G3	wifi_ax [†]	600	5.000
S13	G3	wifi_n	50	14.0
S14	G3	wifi_legacy	10	30.0
S15	G3	nr_5g_embbs [†]	500	7.000
S16	G3	nr_5g_edge	100	14.0
S17	G3	lte_good	50	30.0
S18	G3	lte_poor	10	70.0
S19	G4	satellite_geo	10	320.0

[†] 该场景未纳入图 4 和表 8 详细展示的 15 个代表性配对子集（S1–S8、S10、S13、S14、S16–S19），但仍包含在 36 场景 UDP 突发聚合统计中。编号 S1–S19 为全文统一的场景标识。

4.3 纯 TCP 设置下的传输性能

表 3、表 4 与表 5 分别给出 19 个代表性场景在纯 TCP 设置下的聚合吞吐量、前向流平均单向时延与丢包率（含 Jain 指数）。图 2 与图 3 以对数坐标给出前两项指标（横轴为表 2 中的场景编号）；黑色短划线标出配置的基础单向传播时延，柱体高出短划线的部分近似反映排队及其他仿真处理开销。

表 3：纯 TCP 设置：聚合有效吞吐量 (Mbps，代表性 19 场景)

场景	Swift	NewReno	CUBIC	BBR
intra_rack_10g	9302.6	8687.4	8865.0	8762.9
intra_rack_25g	22425.4	20483.8	21236.8	21161.4
leaf_spine_20g	18016.7	16279.4	17545.7	16805.3
asymmetric_high	878.6	798.6	839.7	817.0
congested_heavy	454.5	406.8	430.3	429.6
symmetric_low	905.0	886.5	886.5	886.5
dc_500m	460.2	430.5	428.8	435.0
dc_100m	86.7	80.1	82.2	81.1
cross_dc_wan	874.6	823.5	826.8	856.1
wan_metro	909.4	844.4	860.4	863.4
wifi_ac	340.5	325.1	332.1	325.7
wifi_ax	522.6	494.6	508.4	497.7
wifi_n	42.7	40.0	41.3	40.8
wifi_legacy	8.6	7.9	8.2	7.6
nr_5g_embbs	429.5	399.0	408.6	402.2
nr_5g_edge	87.8	82.9	82.5	83.1
lte_good	42.3	40.0	41.1	41.1
lte_poor	8.5	7.9	8.1	8.2
satellite_geo	8.8	8.2	8.4	8.3

吞吐量：在 RED/ECN 对称配置下，Swift 在所考察场景中整体呈现吞吐领先。以逐场景 NewReno/CUBIC/BBR 中的最强吞吐基线为参照，36 场景增益范围为 2.1%~5.8%，中位数与均值均约 4.1%。代表性结果包括：wan_longhaul 为 881.24 Mbps，较

表 4：纯 TCP 设置：前向流均值的非加权平均单向时延 (ms，代表性 19 场景)

场景	Swift	NewReno	CUBIC	BBR
intra_rack_10g	0.0502	0.0515	0.0524	0.0513
intra_rack_25g	0.0199	0.0190	0.0207	0.0206
leaf_spine_20g	0.0274	0.0279	0.0331	0.0281
asymmetric_high	0.4194	0.4292	0.4682	0.4203
congested_heavy	0.9561	0.8742	1.0058	0.9892
symmetric_low	0.5031	0.5385	0.5029	0.4958
dc_500m	0.8489	0.9003	0.8516	0.8815
dc_100m	3.9000	3.7952	4.0837	3.8845
cross_dc_wan	5.4327	5.7355	5.4350	5.2814
wan_metro	2.6549	2.7965	2.7151	2.6598
wifi_ac	8.0957	8.1044	8.0509	7.7681
wifi_ax	5.8703	5.8100	5.9325	5.7430
wifi_n	22.2665	23.0975	23.2653	21.9797
wifi_legacy	71.2199	72.2514	79.9873	64.5116
nr_5g_embbs	8.0926	8.2075	7.9536	7.7523
nr_5g_edge	19.3383	19.2767	19.0235	18.2525
lte_good	37.6320	39.5563	39.1253	37.4833
lte_poor	114.4919	116.4265	115.4021	118.1664
satellite_geo	381.2900	378.8589	378.9036	367.2213

CUBIC 高 4.35%；wan_metro 为 909.37 Mbps，较 BBR 高 5.32%；satellite_leo 为 134.95 Mbps，较 BBR 高 4.77%；lte_poor 为 8.50 Mbps，较 BBR 高 3.91%；dc_500m 为 460.17 Mbps，较 BBR 高 5.79%。这些结果覆盖长距离、无线/蜂窝命名参数剖面 and 近端链路，但不外推为真实异构终端硬件上的性能结论。

时延：Swift 的时延总体与基线处于相近量级，但并非普遍最低。按未舍入复算结果，Swift 严格时延最低的场景为 7 个，在 21/36 个场景中逐场景最低时延相差不超过 2%。wan_longhaul 中 Swift 为 26.1603 ms，较 BBR 低 0.9%；wan_metro、lte_poor 与 dc_500m 也呈现近优或最低时延。相反，wifi_legacy、nr_5g_edge 和 satellite_geo 相对最低时延基线分别高 10.4%、5.9% 和 3.8%。这些差异表明吞吐领先与时延表现之间仍存在场景依赖的权衡，不能由单种子结果作统一因果归因。

丢包与公平性：36 场景范围内四协议纯 TCP 丢包率上限为 0.026%。Swift 严格丢包率最低仅见于 8 个场景，因而丢包表现应描述为与基线大体可比，而非系统性最优。Jain 指数均接近 1；Swift 严格 Jain 最优的场景为 11 个，并在 32/36 个场景中距逐场景最优值不超过 0.002。表 6 保留均值与最小值以展示量级，不据此作“最均衡”的绝对结论。

4.3.1 分组解读

G1——微秒级 RTT 近端高速链路 (S1–S3)：瓶颈 10~25 Gbps、基础单向时延 4~9 μ s。该组中 Swift 相对最强基线的吞吐增益为 2.7%~5.6%。四协议时延为 0.019~0.052 ms，差异处于数十微秒量级；这些结果表明该参数组中的吞吐提升未伴随数量级时延变化，但缺少队列轨迹和消融实验，不能进一步作机制因果归因。

G2——近端共享/城域/跨域链路 (S4–S10)：瓶颈 100 Mbps~1 Gbps、基础单向时延 0.012~5.02 ms，包含按 BDP 定尺缓冲与触及 100 包下界的配置。该组整体呈现 2.2%~5.8% 的吞吐增益。wan_metro 中 Swift 吞吐较 BBR 高 5.32%，时延 2.6549 ms 亦接近最低值；dc_500m 吞吐较 BBR 高 5.79%，且时延为该场景最

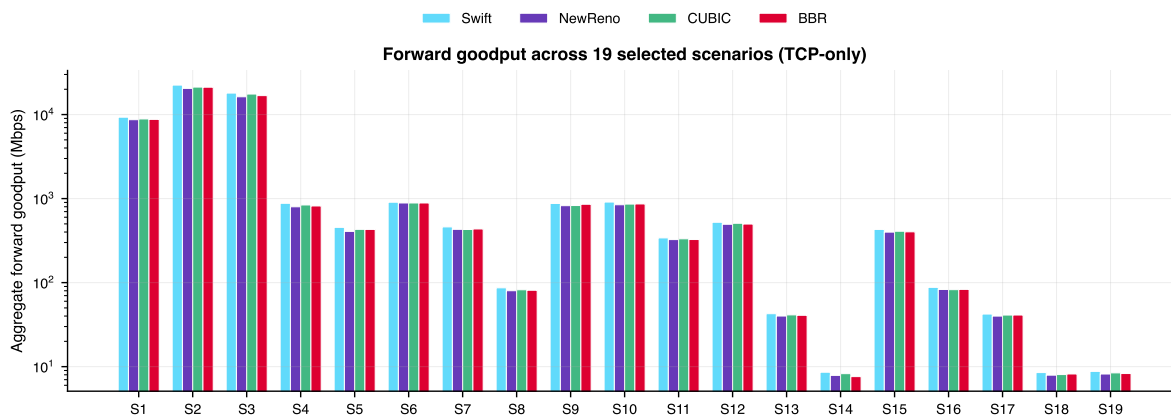


图 2: 19 个代表性场景的聚合前向吞吐量（对数坐标）。该子集中 Swift 整体呈现吞吐领先

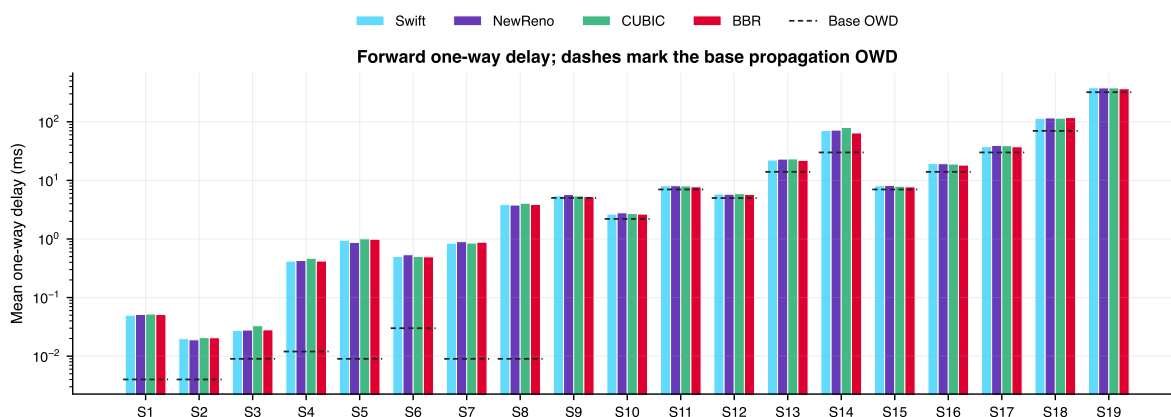


图 3: 19 个场景的前向流平均单向时延（对数坐标；场景值为各流均值的非加权算术平均）。黑色短划线为基础单向传播时延

低。dc_100m 中 Swift 时延 3.9000 ms，较 NewReno 的 3.7952 ms 略高，体现了场景依赖的权衡。

G3——无线与蜂窝接入参数剖面 (S11–S18): 这些名称仅表示点到点链路的带宽与时延配置。该组总体呈现约 2.5%~5.7% 的吞吐增益；lte_poor 中 Swift 吞吐较 BBR 高 3.91%，时延 114.49 ms 为该场景最低。混合结果出现在 wifi_legacy 与 nr_5g_edge，Swift 相对各自最低时延分别高 10.4% 和 5.9%。

G4——GEO 卫星参数剖面 (S19): 基础单向时延 320 ms、瓶颈 10 Mbps。Swift 为 8.78 Mbps，相对最强吞吐基线 CUBIC 的 8.43 Mbps 高 4.15%；其时延 381.29 ms 则比最低的 BBR 367.22 ms 高 3.8%。该结果同时体现长 RTT 配置下的吞吐收益与时延代价。

4.3.2 补充场景：远距离与超高速链路的全矩阵覆盖

除表2的 19 个场景外，全矩阵还包含 17 个补充场景，覆盖 LEO 卫星、长途 WAN、跨 Pod、叶脊扩展、100G 近端、RDMA 类、不同过订阅比例、拥塞梯度和混合流量配置。补充集合总体延续约 2%~6% 的吞吐领先趋势。LEO 卫星参数剖面纯 TCP 为 134.95 Mbps，较 BBR 的 128.80 Mbps 高 4.77%，UDP 突发下为 93.58 Mbps，较 CUBIC 的 88.86 Mbps 高 5.31%；长途 WAN 纯 TCP 为 881.24 Mbps，较 CUBIC 高 4.35%，且时延较 BBR 低 0.9%。超高速参数剖面也显示吞吐提升，但 RDMA 类 50 Gbps 场景在 UDP 突发下相对最低时延高

9.4%，因此不能仅以吞吐指标概括补充集合。

4.4 UDP 突发跨流量鲁棒性评估

手机、笔记本、台式机及其他端点所连接的共享路径可能同时承载突发性非响应式流量。本节在严格 A/B 对照下评估四协议在“平均负载为瓶颈速率 32%、On 期峰值 64%、占空比 50%”的 UDP 突发干扰下的表现（第4.1节），并与纯 TCP 设置形成 36 组场景配对；该负载模拟共享路径上的跨流量，不代表具体终端硬件行为。

以同一场景的纯 TCP 结果为基准，表7给出四种协议在 36 组配对上的吞吐量保持率 T_{udp}/T_{tcp} 、时延膨胀 $D_{udp}/D_{tcp} - 1$ 与新增丢包率。图4按 15 个代表性配对场景展示吞吐量变化与新增丢包。

吞吐量：UDP 突发使四种协议的聚合吞吐量平均保持在纯 TCP 结果的 68.6%~68.8%，其中 Swift 均值为 68.6%，与基线可比。就突发设置中的绝对吞吐量而言，Swift 在所考察场景中多数呈现领先；相对逐场景最强基线的增益范围为 2.2%~5.9%，中位数 3.6%，均值 3.7%。这表明当前单种子矩阵中未观察到其相对吞吐优势被明显削弱，但不能据此声称在其他突发强度或随机种子下必然保持。表8给出 15 个代表性配对场景的绝对吞吐量与时延。

时延、丢包与公平性：UDP 突发下 Swift 严格时延最低的场景为 10 个，在 19/36 个场景中最低时延相

表 5: 纯 TCP 设置: 丢包率 (%) 与 Jain 公平性指数 (代表性 19 场景)

场景	丢包率 (%)				Jain 指数			
	Swift	NewReno	CUBIC	BBR	Swift	NewReno	CUBIC	BBR
intra_rack_10g	0.018	0.021	0.021	0.022	1.000	0.999	1.000	0.999
intra_rack_25g	0.010	0.008	0.011	0.015	0.998	0.999	0.999	0.999
leaf_spine_20g	0.010	0.010	0.018	0.011	1.000	0.998	1.000	0.998
asymmetric_high	0.017	0.011	0.017	0.016	0.999	0.998	0.996	1.000
congested_heavy	0.015	0.015	0.016	0.021	0.999	0.999	0.994	0.989
symmetric_low	0.021	0.020	0.022	0.021	1.000	0.996	0.999	1.000
dc_500m	0.015	0.016	0.015	0.015	0.999	0.999	1.000	1.000
dc_100m	0.008	0.009	0.013	0.009	0.999	1.000	0.999	0.997
cross_dc_wan	0.015	0.011	0.015	0.019	1.000	0.995	0.996	1.000
wan_metro	0.021	0.011	0.023	0.024	1.000	1.000	1.000	0.999
wifi_ac	0.014	0.014	0.014	0.017	0.997	0.999	0.999	0.992
wifi_ax	0.012	0.013	0.013	0.018	1.000	0.999	0.999	1.000
wifi_n	0.013	0.009	0.013	0.013	0.999	0.996	0.997	1.000
wifi_legacy	0.021	0.023	0.022	0.000	0.998	0.995	0.996	0.996
nr_5g_emb	0.012	0.012	0.017	0.013	1.000	0.995	1.000	0.990
nr_5g_edge	0.019	0.020	0.017	0.017	0.999	0.996	0.999	0.993
lte_good	0.013	0.009	0.013	0.017	0.998	0.998	0.991	1.000
lte_poor	0.021	0.022	0.022	0.022	0.999	0.993	0.987	0.998
satellite_geo	0.020	0.022	0.021	0.022	0.997	0.999	0.990	0.999

表 6: 全矩阵 (36 场景) 丢包率与公平性聚合统计

设置	协议	丢包均值	丢包最大	Jain 均值	Jain 最小
纯 TCP $n = 36$	Swift	0.0150	0.0225	0.9989	0.9967
	NewReno	0.0137	0.0227	0.9980	0.9933
	CUBIC	0.0176	0.0237	0.9977	0.9871
	BBR	0.0159	0.0259	0.9981	0.9888
UDP 突发 $n = 36$	Swift	0.0279	0.0435	0.9990	0.9940
	NewReno	0.0269	0.0352	0.9976	0.9925
	CUBIC	0.0315	0.0431	0.9977	0.9877
	BBR	0.0287	0.0420	0.9983	0.9871

丢包率单位为%; Jain 指数接近 1 表示 3 条前向流的吞吐分配接近均衡。均值和最小值仅用于量级比较。

表 7: UDP 突发跨流量鲁棒性聚合指标 (配对场景, $n = 36$)

协议	保持率均值	保持率最小	时延膨胀均值	新增丢包均值
Swift	68.6%	62.4%	+2.9%	+0.013 pp
NewReno	68.6%	62.8%	+3.1%	+0.013 pp
CUBIC	68.8%	63.9%	+2.2%	+0.014 pp
BBR	68.7%	62.1%	+3.3%	+0.013 pp

保持率与时延膨胀按逐场景配对后取均值/最小值; 新增丢包 = 突发设置丢包率 - 纯 TCP 设置丢包率。

差不超过 2%; 严格 Jain 最优的场景为 17 个, 在 35/36 个场景中距最优值不超过 0.002; 严格丢包率最低仅见于 10 个场景。总体上, 公平性和丢包与基线接近, 而非统一占优。混合时延结果包括 congested_medium、nr_5g_edge 和 rdma_like_50g, Swift 相对最低时延分别高 12.2%、7.9% 和 9.4%。

远距离场景在突发下的表现: GEO 卫星参数剖面 (S19) 中 Swift 为 5.65 Mbps, 较最强吞吐基线 BBR 高 3.29%; LEO 卫星参数剖面中 Swift 为 93.58 Mbps, 较 CUBIC 高 5.31%。wan_metro 在突发下为 591.17 Mbps, 较 BBR 高 2.69%, 且时延接近该场景最低值。这些结果说明所测长 RTT 与城域参数剖面中仍观察到吞吐收益; 由于没有机制消融、更多随机种子或真实链路实验, 不能将该现象单独归因于某一控制模块。

4.5 机制一致性与证据边界

实现一致的解释: Swift 在 ECN、普通丢包和超时路径上分别使用 0.75、0.70 和 0.50 的保留因子, 并结合 4 个 ACK 事件冻结、滑动时间窗累计 ACK 投递速率、40 样本最大值滤波以及 $\alpha \times BDP$ 目标窗口跟踪。这些机制与“在 RED/ECN 环境下保留较充分的发送窗

口, 同时限制重复缩减和单 ACK 速率噪声”的解释相容。相比之下, NewReno、CUBIC 与 BBR 具有不同的控制律。本文未测量逐协议在途窗口与队列轨迹, 也未执行逐机制关闭的消融仿真, 因此不能由聚合指标证明某一机制是吞吐差异的唯一原因, 也不对基线内部状态作未测量的因果判断。

混合结果与外推边界: (1) Swift 严格丢包率最优的场景在纯 TCP 和 UDP 突发下分别为 8 个和 10 个, 故丢包只能描述为整体可比; (2) 纯 TCP 下 wifi_legacy、nr_5g_edge 和 satellite_geo 存在 3.8%~10.4% 的相对时延抬升, UDP 突发下 congested_medium、nr_5g_edge 和 rdma_like_50g 存在 7.9%~12.2% 的抬升; (3) Swift 的吞吐保持率均值 68.6% 与基线 68.6%~68.8% 相当。上述观察仅对应按瓶颈速率比例缩放单种子 UDP 负载, 不外推到其他突发强度、队列管理方式或真实异构终端。

5 讨论

5.1 实现性质与定性分析

有界更新性质: 在拥塞避免阶段, 窗口以 $\max(\gamma MSS, MSS, (target - cwnd)/16)$ 向上调整, 或以不超过当前超出量的规则向 $\alpha \times BDP$ 回落; 最终窗口再钳位到 $[4MSS, \max(4BDP, 200MSS)]$ 。检测到 ECN、普通丢包或超时, 窗口分别按 0.75、0.70 和 0.50 的保留因子处理, 连续缩减计数与 4 个 ACK 事件冻结限制重复响应。上述性质可由代码直接检查, 但本文未给出 Lyapunov 收敛证明, 也不声称目标窗口在任意网络条件下都是稳定平衡点。

复杂度与计算开销: 当前实现采用确定性规则。带宽样本与 RTT 样本队列容量均为 40, 累计 ACK 时间窗队列容量为 4096, 其余状态为固定数量的计数器和标量; 因此在这些固定上限下, 单次决策和每流存储开销均有界。窗口最大值搜索及时间窗清理的实际代价取决于队列内样本数, 本文不将其简化为未经实测的固定运行时。仓库未保存独立的决策时延、CPU 利用率或内存压测日志, 故不报告具体硬件上的性能开销。

解释边界: 超时、普通丢包与 ECN 的优先级及保

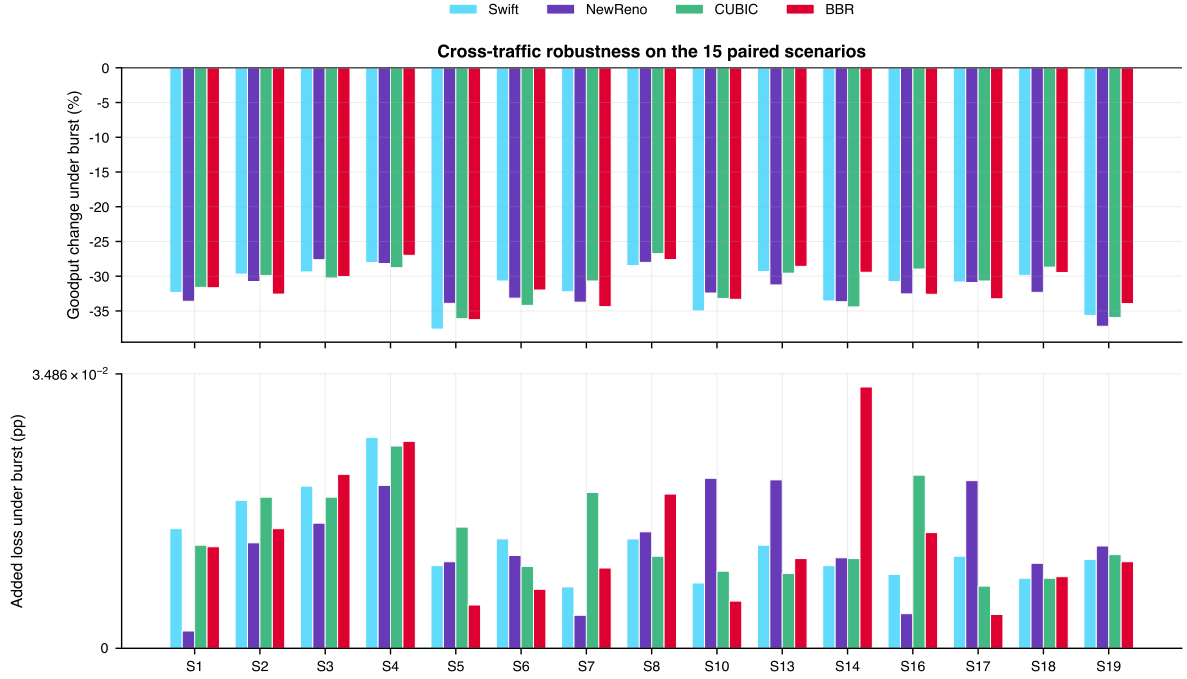


图 4: 15 个代表性配对场景的跨流量鲁棒性: UDP 突发引起的吞吐量变化 (上) 与新增丢包 (下, 符号对数坐标)

表 8: UDP 突发设置下 15 个代表性场景的吞吐量与时延 (吞吐量为前向流之和, 时延为流均值的非加权平均)

场景	吞吐量 (Mbps)				平均单向延迟 (ms)			
	Swift	NewReno	CUBIC	BBR	Swift	NewReno	CUBIC	BBR
intra_rack_10g	6295.2	5767.8	6063.1	5988.7	0.048	0.048	0.054	0.049
intra_rack_25g	15765.9	14181.8	14888.4	14268.4	0.020	0.020	0.023	0.020
leaf_spine_20g	12721.9	11782.0	12236.4	11754.9	0.030	0.031	0.034	0.030
asymmetric_high	632.3	573.6	598.1	596.5	0.556	0.504	0.567	0.563
congested_heavy	283.5	268.9	274.9	273.9	0.864	0.829	0.857	0.855
symmetric_low	627.2	592.5	583.3	602.9	0.469	0.488	0.469	0.516
dc_500m	311.8	285.3	297.2	285.4	0.908	0.890	0.957	0.862
dc_100m	62.0	57.6	60.2	58.7	4.224	4.362	4.832	4.229
wan_metro	591.2	570.6	574.7	575.7	2.609	2.747	2.625	2.693
wifi_n	30.1	27.5	29.1	29.2	23.133	22.768	23.941	22.503
wifi_legacy	5.7	5.2	5.4	5.4	73.369	73.677	75.589	74.655
nr_5g_edge	60.8	55.9	58.6	56.0	19.610	19.561	19.372	18.176
lte_good	29.2	27.6	28.5	27.5	38.549	38.993	39.322	37.507
lte_poor	6.0	5.4	5.7	5.8	117.590	111.634	123.046	122.328
satellite_geo	5.7	5.1	5.4	5.5	361.464	381.743	354.523	363.152

留因子表达了工程化的响应强度排序。本文没有估计这些信号的条件概率、似然比或最优控制输入, 也没有形式化证明连续缩减保护的收敛性; 相关机制仅按实现行为和聚合仿真结果作定性讨论。

5.2 与相关工作的详细对比

vs. Gemini [24]: 两者均保留可解释的白盒控制逻辑, 但参数适配方式、决策位置和时间尺度不同。Gemini 的 Fusion 固定在内核数据面, Booster 在带外以分钟级、流组粒度用贝叶斯优化搜索参数; Swift 不执行学习或搜索, 而是在协议栈外按每个窗口调整事件运行确定性启发式规则, 并用 RTT 与基线相对反馈调节每流 α 。Swift 还能读取 ECN 与 CA 状态机, 以代码中的优先级规则区分 ECE/CWR 相关收缩、普通丢包和超时; 其代价是每事件一次跨进程同步往返, 当前形态适用于研究与仿真平台, 尚不具备 Gemini 已有的生产部署证据。

vs. Aurora [10]: Swift 采用 11 维有效状态子空间 (Aurora 常用 10 维观测), 并引入 ECN 和 CA 状态参数。Swift 采用基于规则的确定性决策而非深度神经网络, 无需训练即可运行, 决策路径可逐分支解释。Aurora 的优势在于神经网络的表示能力, 代价是训练数据收集、模型更新与推理开销; Swift 则以可解释规则和协议栈内部信号换取工程可控性。本文不主张 Swift 在策略表示能力上优于 Aurora。

vs. DCTCP [8]: Swift 直接读取 ECN 协议栈状态进行离散分类, 而 DCTCP 依据一个 RTT 内的标记比例更新拥塞估计并按 $1 - \alpha/2$ 缩窗。Swift 同时使用丢包、ECN 与 CA 状态, 并设置 0.70/0.75/0.50 三类保留因子; 其 D_{max} 与冻结计数是限制连续离散响应的工程机制。本文未在当前实验矩阵中纳入 DCTCP, 因而不作二者性能优劣结论。

vs. BBR [5]: Swift 综合 ECN、CA 状态与 RTT 等

表 9: Swift 与主要相关算法的特性对比

特性	Swift	Gemini	Aurora	DCTCP
有效状态维度	11	流级统计	10	N/A
ECN 状态利用	是	否	否	是
CA 状态利用	是	否	否	否
决策机制	规则	规则	DNN	公式
在线调整对象	参数	参数	策略	固定状态量
自适应粒度	每流/每事件	流组/分钟级	每决策	每 RTT
决策位置	栈外进程	内核数据面	栈外/栈内	内核
策略训练	无	无 (贝叶斯搜索参数)	需要	无
差异化响应	是	否	否	比例 ECN 响应
生产部署证据	无	有	无	有

信号, 并以 $\alpha \times BDP$ 有界目标窗口调整 cwnd; BBR 则依赖瓶颈带宽、最小 RTT 估计和周期性增益循环。在所考察场景中, Swift 相对逐场景最强吞吐基线的纯 TCP 增益为 2.1%~5.8%, 而 UDP 突发下 Swift 与 BBR 的吞吐保持率均值分别为 68.6% 和 68.7%。时延结果更为混合: Swift 在 21/36 个纯 TCP 场景中距最低值不超过 2%, 但 wifi_legacy、nr_5g_edge 和 satellite_geo 相对最低时延仍高 3.8%~10.4%。本文据此仅报告实现差异与观测结果, 不推断未记录的内部机制因果关系。

5.3 适用边界与未来方向

在途窗口与时延的剩余代价: Swift 以 $\alpha \times BDP$ 为目标且 α 上界为 1.30, 部分配置中同时观察到吞吐提升与相对时延上升。纯 TCP 下 wifi_legacy、nr_5g_edge 和 satellite_geo 相对最低时延分别高 10.4%、5.9% 和 3.8%; UDP 突发下 congested_medium、nr_5g_edge 和 rdma_like_50g 分别高 12.2%、7.9% 和 9.4%。由于没有窗口/队列消融数据, 本文不把这些差异唯一归因于 α 。可探索的方向包括引入排队时延估计 $q_{est} = RTT_{smoothed} - RTT_{min}$ 、在持续 RTT 膨胀时收紧 α , 以及采用多时间尺度 BDP 估计。

参数预设依赖: 保留因子 (0.70/0.75/0.50)、 α 边界 ([0.85, 1.30])、连续递减阈值 ($D_{max} = 3$) 与降窗后冻结的 ACK 事件数 (4) 均为手动调优的预设值。未来可通过元学习 [27] 或如 Gemini [24] 所示的带外贝叶斯优化实现参数的在线搜索, 二者可与本文的每事件决策回路组合成分层结构。

深度/学习型策略增强: 当前基于规则的启发式策略可解释性强但表示能力有限。后续若引入学习型组件 (如离线强化学习训练、再经模型蒸馏提取规则), 须在完整实验矩阵与异常过滤流程上重新评估, 本文不将其作为当前系统的组成部分。

部署代价与真实环境验证: 每个窗口调整事件一次跨进程往返是当前实现的主要部署障碍。后续需将决策逻辑内联为 Linux 内核 TCP 模块或 QUIC 用户态拥塞控制插件, 并在覆盖无线/蜂窝接入、广域网与卫星链路的真实或半实物测试床上验证。仿真与真实环境的差异 (NIC offload、中断合并、NUMA 效应、终端操作系统调度与无线链路波动) 可能显著影响实际性能。

证据边界说明: 本文的测量来自 36 个链路参数剖

面、4 种协议和纯 TCP/UDP 突发两类设置下的 ns-3.40 FlowMonitor 前向流统计。每个配置只有一个随机运行号, 不提供置信区间或显著性检验; 场景采用 RED/ECN 对称配置并通过点到点链路参数抽象 WiFi、蜂窝、WAN 和卫星条件, 不是异构终端硬件或真实接入协议验证。现有结果支持“在所考察场景中整体呈现约 2%~6% 吞吐领先、时延/丢包/公平性表现混合且总体可比”的表述, 不支持普遍最优结论。

6 结论

本文提出了 Swift——一种面向远距离传输、手机/笔记本/台式机异构端点及多样接入与路径条件的启发式拥塞控制方法。其 Python 控制器经 ns3-gym/OpenGym 同步通道接收 ns-3 协议栈状态, 并在 GetSsThresh 与 IncreaseWindow 事件上返回 ssthresh/cwnd 动作; 该过程由确定性白盒规则完成, 不依赖策略训练。核心技术贡献保持为三点: (1) 面向远距离与异构端点的多信号拥塞状态感知, 联合 ECN、CA 状态、RTT、最小 RTT 和在途字节等信息区分超时、ECN 与普通丢包; (2) ns3-gym 集成的事件驱动启发式融合控制, 以滑动时间窗累计 ACK 投递速率和最小 RTT 估计 BDP, 通过 RTT、基线相对反馈与增长趋势调节每流 α 并有界跟踪目标窗口; (3) 面向不确定拥塞信号的稳定性与安全机制, 采用 0.70/0.75/0.50 差异化保留因子、至多 3 次连续缩减、4 个 ACK 事件冻结、4MSS 下界、 $\max(4BDP, 200MSS)$ 上界以及 CA_LOSS 下的陈旧动作失效。

在 ns-3.40 平台上, 本文以 36 个链路/路径参数剖面、4 种协议和纯 TCP/UDP 突发两种设置形成 288 条记录。纯 TCP 下, Swift 在所考察场景中整体呈现吞吐领先, 相对逐场景最强基线的增益为 2.1%~5.8% (中位数与均值均约 4.1%); 严格时延最低的场景为 7 个, 21/36 个场景与最低时延相差不超过 2%; 严格 Jain 最优为 11 个, 32/36 个场景距最优值不超过 0.002; 严格丢包率最优仅为 8 个。UDP 突发下吞吐增益为 2.2%~5.9% (均值 3.7%), 吞吐保持率均值 68.6% 且与基线可比; 严格最低时延、严格 Jain 最优和严格丢包率最低的场景分别为 10、17 和 10 个。wan_longhaul、wan_metro、satellite_leo、lte_poor 和 dc_500m 提供了吞吐或吞吐-时延均较有利的代表性结果。

证据边界同样明确: wifi_legacy、nr_5g_edge、satellite_geo 以及 UDP 突发下的 congested_medium、rdma_like_50g 等场

景存在 3.8%~12.2% 的相对时延抬升，公平性和丢包总体接近基线而非统一最优。每配置仅有一个随机种子，未开展消融、置信区间或显著性检验；场景名仅代表点到点链路参数剖面，不构成异构终端硬件级验证。后续工作包括多种子重复实验、队列/窗口轨迹与机制消融、突发强度扫描、真实或半实物端点测试，以及将每事件规则内联到内核或用户态协议栈以降低跨进程开销。

参考文献

- [1] S. Floyd, T. Henderson, and A. Gurtov, “The NewReno modification to TCP’s fast recovery algorithm,” *RFC 6582*, 2012.
- [2] S. Ha, I. Rhee, and L. Xu, “CUBIC: a new TCP-friendly high-speed TCP variant,” *ACM SIGOPS Operating Systems Review*, vol. 42, no. 5, pp. 64–74, 2008.
- [3] J. Gettys and K. Nichols, “Bufferbloat: Dark buffers in the Internet,” *Communications of the ACM*, vol. 55, no. 1, pp. 57–65, 2012.
- [4] L. S. Brakmo and L. L. Peterson, “TCP Vegas: End to end congestion avoidance on a global Internet,” *IEEE Journal on Selected Areas in Communications*, vol. 13, no. 8, pp. 1465–1480, 1995.
- [5] N. Cardwell, Y. Cheng, C. S. Gunn *et al.*, “BBR: Congestion-based congestion control,” *ACM Queue*, vol. 14, no. 5, pp. 20–53, 2016.
- [6] M. Hock, R. Bless, and M. Zitterbart, “Experimental evaluation of BBR congestion control,” in *2017 IEEE 25th International Conference on Network Protocols (ICNP)*. IEEE, 2017, pp. 1–10.
- [7] K. Ramakrishnan, S. Floyd, and D. Black, “The addition of explicit congestion notification (ECN) to IP,” *RFC 3168*, 2001.
- [8] M. Alizadeh, A. Greenberg, D. A. Maltz *et al.*, “Data center TCP (DCTCP),” *ACM SIGCOMM Computer Communication Review*, vol. 40, no. 4, pp. 63–74, 2010.
- [9] N. Jay, N. Rotman, B. Godfrey *et al.*, “A deep reinforcement learning perspective on Internet congestion control,” in *International Conference on Machine Learning*. PMLR, 2019, pp. 3050–3059.
- [10] —, “Internet congestion control via deep reinforcement learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [11] S. Abbasloo, C.-Y. Yen, and H. J. Chao, “Classic meets modern: A pragmatic learning-based congestion control for the Internet,” in *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication*. ACM, 2020, pp. 632–647.
- [12] C.-Y. Yen, S. Abbasloo, and H. J. Chao, “Computers can learn from the heuristic designs and master internet congestion control,” in *Proceedings of the ACM SIGCOMM 2023 Conference*. ACM, 2023, pp. 255–274.
- [13] H. Yu, J. Zheng, Z. Du, Y. Liu, B. Quan, T. Chen, and G. Chen, “When classic meets intelligence: A hybrid multipath congestion control framework,” *IEEE/ACM Transactions on Networking*, vol. 32, no. 4, pp. 3575–3590, 2024.
- [14] P. Gawłowicz and A. Zubow, “ns-3 meets OpenAI Gym: The playground for machine learning in networking research,” in *Proceedings of the 22nd International ACM Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems*. ACM, 2019, pp. 113–120.
- [15] V. Jacobson, “Congestion avoidance and control,” *ACM SIGCOMM Computer Communication Review*, vol. 18, no. 4, pp. 314–329, 1988.
- [16] V. Arun and H. Balakrishnan, “Copa: Practical delay-based congestion control for the Internet,” in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*. USENIX Association, 2018, pp. 329–342.
- [17] K. Tan, J. Song, Q. Zhang *et al.*, “A compound TCP approach for high-speed and long distance networks,” in *Proceedings IEEE INFOCOM 2006*. IEEE, 2006, pp. 1–12.
- [18] S. Liu, T. Başar, and R. Srikant, “TCP-Illinois: A loss-and delay-based congestion control algorithm for high-speed networks,” *Performance Evaluation*, vol. 65, no. 6-7, pp. 417–440, 2008.
- [19] S. Mascolo, C. Casetti, M. Gerla *et al.*, “TCP Westwood: Bandwidth estimation for enhanced transport over wireless links,” in *Proceedings of the 7th Annual International Conference on Mobile Computing and Networking*. ACM, 2001, pp. 287–297.
- [20] Y. Li, R. Miao, H. H. Liu *et al.*, “HPCC: High precision congestion control,” in *Proceedings of the ACM Special Interest Group on Data Communication*. ACM, 2019, pp. 44–58.
- [21] Z. Wan, J. Zhang, H. Wei, Z. Jiang, X. Zhong, W. Wu, H. Zhou, T. Pan, and T. Huang, “RECC: Joint congestion control based on RTT and ECN for high-speed RDMA networks,” *Proceedings of the ACM on Networking*, vol. 2, no. CoNEXT4, pp. 1–18, 2024.
- [22] M. Dong, Q. Li, D. Zarchy *et al.*, “PCC: Re-architecting congestion control for consistent high performance,” in *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*. USENIX Association, 2015, pp. 395–408.

- [23] K. Winstein and H. Balakrishnan, “TCP ex machina: Computer-generated congestion control,” *ACM SIGCOMM Computer Communication Review*, vol. 43, no. 4, pp. 123–134, 2013.
- [24] W. Yang, Y. Liu, C. Tian, J. Jiang, and L. Guo, “Gemini: Divide-and-conquer for practical learning-based internet congestion control,” in *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications*. IEEE, 2023, pp. 1–10.
- [25] G. F. Riley and T. R. Henderson, “The ns-3 network simulator,” in *Modeling and Tools for Network Simulation*. Springer, 2010, pp. 15–34.
- [26] R. K. Jain, D.-M. W. Chiu, and W. R. Hawe, “A quantitative measure of fairness and discrimination,” Eastern Research Laboratory, Digital Equipment Corporation, Tech. Rep., 1984.
- [27] C. Finn, P. Abbeel, and S. Levine, “Model-agnostic meta-learning for fast adaptation of deep networks,” in *International Conference on Machine Learning*. PMLR, 2017, pp. 1126–1135.